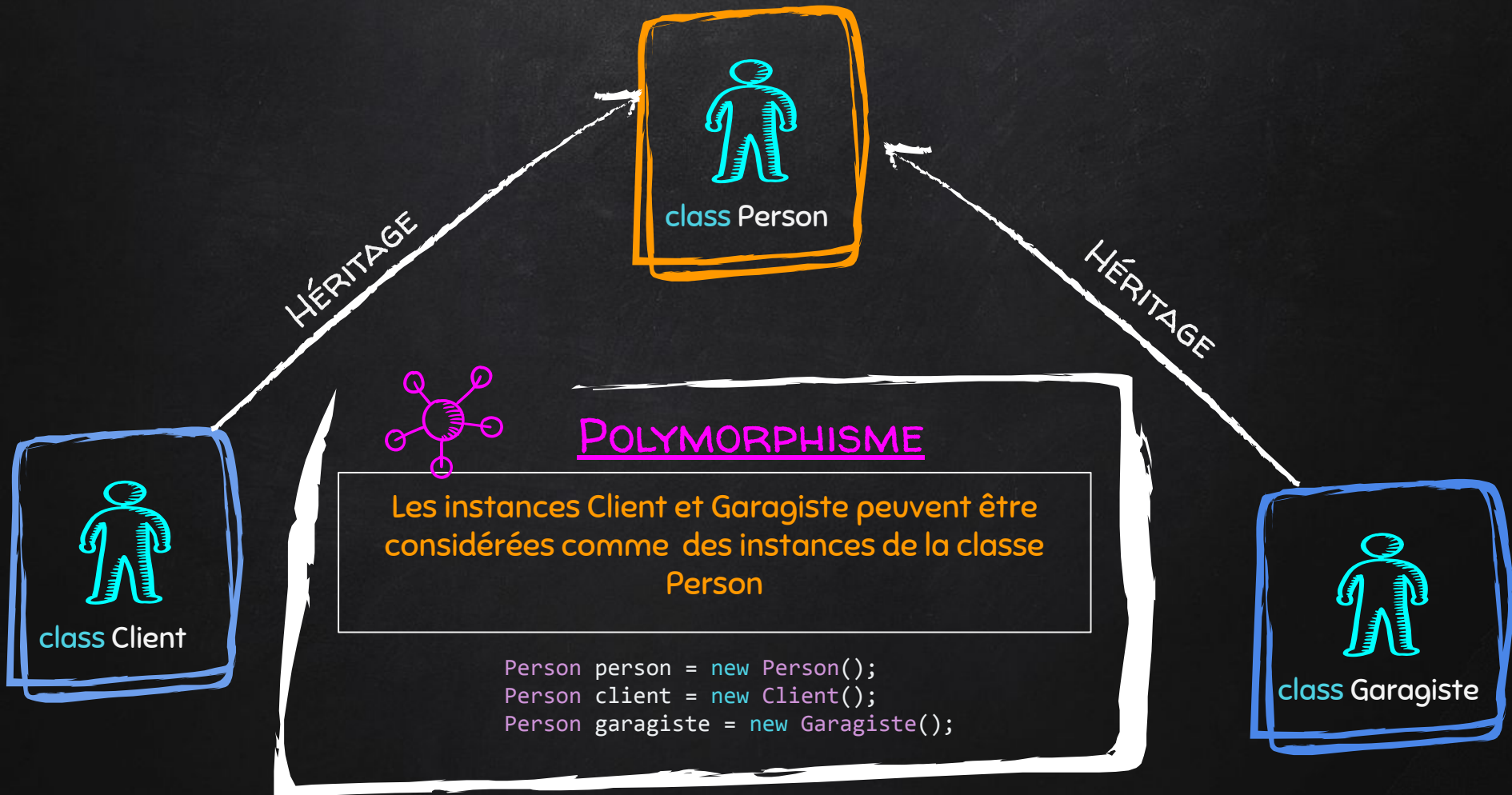




POLYMORPHISME

Polymorphisme et méthodes de substitution



```

class Person // Classe parente
{
    public void Informations()
    {
        Console.WriteLine("Je suis une personne !!!");
    }
}

class Client : Person // Classe enfant
{
    public void Informations()
    {
        Console.WriteLine("Je suis un client !!!");
    }
}

class Garagiste : Person // Classe enfant
{
    public void Informations()
    {
        Console.WriteLine("Je suis un garagiste !!!");
    }
}

```

SUBSTITUTIONS SANS VIRTUAL ET OVERRIDE



A SAVOIR !

La méthode est redéfinie mais n'est pas appelée si on utilise le polymorphisme.

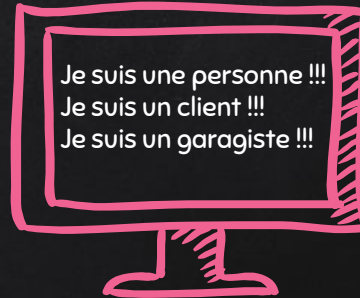
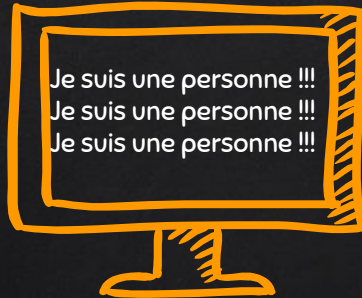
C'est la méthode de la classe parente qui est appelée !

```

class Program
{
    static void Main(string[] args)
    {
        Person person = new Person();
        Person client = new Client();
        Person garagiste = new Garagiste();

        person.Informations();
        client.Informations();
        garagiste.Informations();
    }
}

```



```

class Program
{
    static void Main(string[] args)
    {
        Person person = new Person();
        Client client = new Client();
        Garagiste garagiste = new Garagiste();

        person.Informations();
        client.Informations();
        garagiste.Informations();
    }
}

```

SUBSTITUTIONS AVEC VIRTUAL ET OVERRIDE

```
class Person // Classe parente
{
    public virtual void Informations()
    {
        Console.WriteLine("Je suis une personne !!!");
    }
}

class Client : Person // Classe enfant
{
    public override void Informations()
    {
        Console.WriteLine("Je suis un client !!!");
    }
}

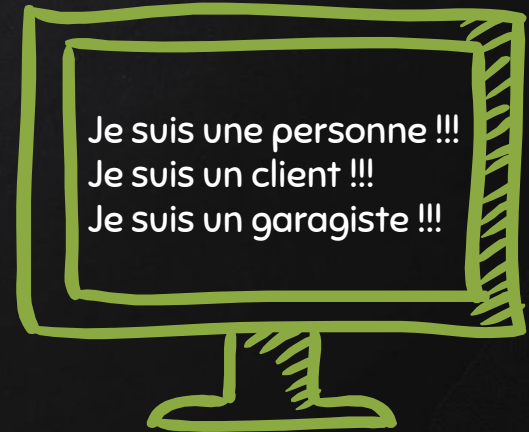
class Garagiste : Person // Classe enfant
{
    public override void Informations()
    {
        Console.WriteLine("Je suis un garagiste !!!");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Person person = new Person();
        Person client = new Client();
        Person garagiste = new Garagiste();

        person.Informations();
        client.Informations();
        garagiste.Informations();
    }
}
```



Le mot clé **virtual** dans une classe parente permet de redéfinir la méthode dans les classes enfants avec le clé **override** !



ENNONCÉ

1. Créer une nouvelle classe Vehicle
2. Créer une classe Truck qui hérite de Vehicle et faire hériter la classe Car de Vehicle
3. Permettre au garage d'avoir la liste des véhicules actuellement en réparation
4. Afficher la personne en charge de réparer le véhicule
5. Ajouter une propriété dommage au véhicule et skills au garagiste en pourcentage et estimé le temps de réparation:
 - voiture : $1\% = 1h30 * (2 - (skills/100))$
 - camion : $1\% = 2h * (2 - (skills/100))$



TP 5



UTILISER

- Héritage
- Polymorphisme
- Substitution
- Utilisation de **base**
- Classe
- Méthodes
- Accesseurs